

Clase 2: ETL con Python

Módulo 7: Data Pipelines y plataformas de visualización

Claudio Torres Fonseca

mail: `claudio.torresf@usm.cl`

Universidad de Santiago de Chile

Diplomado en Data Engineer
2026

Contenido

- 1 The Beam Model
- 2 Transformaciones
- 3 Streaming
- 4 Práctica

¿Qué es Apache Beam?

Definición

- Un modelo de programación unificado de código abierto para definir y ejecutar flujos de procesamiento de datos (Data Pipelines).
- **Batch + strEAM.**
- **Visión:** 'Escribe una vez, ejecuta en cualquier lugar'.
- Separación de Conceptos:
 - **SDK:** El lenguaje que usas (Python, Java, Go).
 - **Runner:** El motor que lo ejecuta (Spark, Flink, Google Cloud Dataflow).

El Modelo de las 4 Preguntas (The Beam Model)

Para diseñar cualquier pipeline, un Data Engineer debe responder:

What/Qué

- ¿Qué resultados se están calculando?
- Transformaciones

Where/Dónde

- ¿En qué tiempo del evento se calculan?
- Windowing

El Modelo de las 4 Preguntas (The Beam Model)

Para diseñar cualquier pipeline, un Data Engineer debe responder:

When/Cuándo

- ¿En qué tiempo del procesamiento se emiten los resultados?
- Triggers

How/Cómo

- ¿Cómo se relacionan los refinamientos de datos?
- Acumulación

El Modelo de las 4 Preguntas (The Beam Model)

Caso QuickRide (una app de transporte tipo Uber)

Caso

- Imagina que queremos calcular cuánto dinero gana un conductor por hora, pero tenemos un problema: los conductores pasan por túneles o zonas con mala señal, y sus datos de GPS y cobro llegan con retraso.

Escenario: El Conductor en el Túnel de la Costanera Norte

- El conductor inicia un viaje a las **10:55** y lo termina a las **11:05**. Sin embargo, debido a la mala señal, el servidor de la nube no recibe la confirmación del pago hasta las **11:15**.

El Modelo de las 4 Preguntas (The Beam Model)

Caso QuickRide (una app de transporte tipo Uber)

What? (¿Qué resultados se calculan?)

- **Definición:** La lógica de negocio o transformación.
- **Ejemplo:** Queremos la **Suma (Sum)** de los montos de los viajes.
- **En Beam:** `beam.CombinePerKey(sum)`.

El Modelo de las 4 Preguntas (The Beam Model)

Caso QuickRide (una app de transporte tipo Uber)

Where? (¿En qué tiempo del evento se calculan?)

- **Definición:** Define cómo agrupamos los datos en bloques de tiempo (Ventanas).
- **Ejemplo:** Usamos Ventanas Fijas (Fixed Windows) de 1 hora.
- **Decisión:** El viaje de las 10:55 pertenece a la ventana de 10:00 a 11:00, aunque el dato llegue físicamente a las 11:15.
- **En Beam:** `beam.WindowInto(FixedWindows(3600))`.

El Modelo de las 4 Preguntas (The Beam Model)

Caso QuickRide (una app de transporte tipo Uber)

When? (¿En qué tiempo de procesamiento se emiten?)

- **Definición:** Determina cuándo el sistema "suelta" el resultado hacia el tablero del gerente o la base de datos (Triggers).
- **Ejemplo:**
 - Podemos emitir un resultado parcial a las 11:00 (cuando cierra la hora).
 - Podemos emitir una corrección a las 11:16, justo después de que llegó el dato retrasado del túnel.
- **En Beam:** `AfterWatermark(...)`.

El Modelo de las 4 Preguntas (The Beam Model)

Caso QuickRide (una app de transporte tipo Uber)

How? (¿Cómo se relacionan las refinaciones de datos?)

- **Definición:** Define qué hacer si llega un dato nuevo para una ventana que ya “cerramos”.
- **Ejemplo:**
 - **Acumulación (Accumulating):** Si a las 11:00 calculamos \$5.000 y a las 11:15 llega el viaje del túnel por \$3.000, el nuevo reporte dirá \$8.000.
 - **Descarte (Discarding):** Simplemente ignoramos el dato que llegó tarde (malo para las finanzas, bueno para estadísticas de tráfico rápidas).
- **En Beam:** `.accumulatingFiredPanels()` o `.discardingFiredPanels()`.

Definiciones Fundamentales

- **Pipeline:** El grafo dirigido acíclico (DAG) que contiene todas las etapas de tu tarea, desde la lectura hasta la escritura.
- **PCollection:** Un set de datos inmutable que puede ser finito (Batch) o infinito (Streaming). No puedes modificar una PCollection; aplicas una transformación y obtienes una nueva.
- **PTransform:** Una operación de procesamiento. Es el "paso" que transforma una PCollection de entrada en una de salida.

Documentación oficial

Links

- **Guía oficial:**
 - <https://beam.apache.org/documentation/programming-guide/>
- **Todos los recursos de aprendizaje:**
 - <https://beam.apache.org/get-started/resources/learning-resources/>
- **Guía interactiva:**
 - <https://tour.beam.apache.org/tour/python/introduction/guide>

Documentación oficial

Links

- **Guías Python:**

- <https://beam.apache.org/get-started/quickstart/python/>

- **Ejemplos:**

- https://github.com/apache/beam/tree/master/sdks/python/apache_beam/examples
- <https://github.com/apache/beam-starter-python/tree/main>

Transformación 1 - Map (Uno a Uno)

Definición

- Toma un elemento y devuelve exactamente un elemento transformado.

Ejemplo Práctico: Convertir precios de CLP a USD.

- Entrada: 80000 (CLP) – > Salida: 82.30 (USD)

```
precios_usd = precios_clp
| "Convertir a USD" >> beam.Map(
    lambda x: x / 972
)
```

Transformación 2 - Filter (Decisión)

Definición

- Evalúa una condición booleana para cada elemento. Si es True, el elemento pasa; si es False, se descarta.

Ejemplo Práctico: Quedarse solo con transacciones de 'Santiago'.

- Entrada: {'ciudad': 'Santiago', ...}, {'ciudad': 'Concepción', ...}
- Salida: Solo el registro de Santiago.

```
solo_santiago = ventas  
| "Filtrar Ciudad" >> beam.Filter(  
    lambda x: x['ciudad'] == 'Santiago'  
)
```

Transformación 3 - FlatMap (Uno a Muchos)

Definición

- Toma un elemento y puede devolver cero, uno o muchos elementos. Útil para 'explotar' listas o limpiar registros.

Ejemplo Práctico: Separar una cadena de texto en palabras individuales.

- Entrada: "Hola Mundo Beam"
- Salida: "Hola", "Mundo", "Beam"

```
palabras = lineas
    | "Tokenizar" >> beam.FlatMap(
        lambda x: x.split(" ")
    )
```


Transformación 4 - Combine / GroupByKey (Agregación)

Definición

- Agrupa elementos que comparten una misma clave o combina todos los elementos en un valor único (como una suma).

Ejemplo Práctico: Calcular el total de ventas por país.

- Entrada: ('Chile', 100), ('Chile', 50), ('Peru', 200)
- Salida: ('Chile', 150), ('Peru', 200)

```
totales = ventas_pares  
| "Sumar por Pais" >> beam.CombinePerKey(sum)
```

Anatomía de un Script de Producción

Ejemplo

- https://github.com/ctorresf/developer-guides/blob/main/apache_beam/vscode-devcontainer-basic-beam-examples/src/simple_pipeline.py
- https://github.com/ctorresf/developer-guides/blob/main/apache_beam/vscode-devcontainer-basic-beam-examples/src/word_count_map.py

Ejecución Multi-Nube: Del Local al Cluster

Desarrollo: DirectRunner (Local)

- **Entorno:** Tu laptop o una máquina virtual simple.
- **Uso:** Pruebas unitarias y validación de transformaciones con muestras pequeñas de datos (ej. 1,000 filas).
- **Limitación:** No escala. Si te quedas sin RAM, el pipeline falla.

Ejecución Multi-Nube: Del Local al Cluster

Producción: Runners Administrados (Cloud) (1/2)

Nube	Runner Recomendado	Características Clave
GCP	Dataflow	Totalmente Serverless. Auto-escalado dinámico y optimización de grafo automática. Es el "hogar natural" de Beam.
AWS	Amazon Kinesis / Amazon Elastic MapReduce (EMR)	Usa el SparkRunner o FlinkRunner sobre instancias de EMR. Requiere más configuración de infraestructura (nodos, memoria).

Ejecución Multi-Nube: Del Local al Cluster

Producción: Runners Administrados (Cloud) (1/2)

Nube	Runner Recomendado	Características Clave
Azure	HDInsight	Utiliza el SparkRunner. Ideal si la empresa ya tiene su ecosistema en Microsoft y usa Delta Lake.
On-Premise	Apache Flink / Spark	Ideal para empresas que por regulación no pueden subir datos a la nube. Tú administras el hardware.

Caso Práctico: Red de Energía Inteligente 'Voltio-Chile'

Contexto del Negocio

- **Voltio-Chile** gestiona una red de 500,000 medidores inteligentes en hogares y empresas. El sistema actual recibe lecturas de consumo cada 15 minutos, pero no puede detectar **picos de sobrecarga** ni **fraudes** (medidores que marcan 0 durante horas de alto consumo) de forma inmediata.

Caso Práctico: Red de Energía Inteligente 'Voltio-Chile'

Enunciado

Tu misión es diseñar e implementar un pipeline que:

- 1 **Identifique Alertas:** Notifique si un medidor supera los 10 kWh (Posible cortocircuito o sobrecarga).
- 2 **Detecte Anomalías:** Identifique medidores con consumo 0.0 en horarios activos.
- 3 **Prepare el Historial:** Limpie y normalice los datos para el reporte de facturación mensual.

Mi Primer Pipeline con Apache Beam

Datos de ejemplo - CSV

```
id_meter , timestamp , consumption_kwh , voltage , status  
M-001,2026-04-02T18:00:00Z,2.5,220,OK  
M-002,2026-04-02T18:00:05Z,12.8,218,OK  
M-001,2026-04-02T18:15:00Z,2.1,221,OK  
M-003,2026-04-02T18:15:10Z,0.0,220,CHECK  
M-999,,4.5,220,OK  
M-002,2026-04-02T18:15:15Z,NULL,215,OK
```

Errores incluidos

Sobrecarga (M-002), Consumo Cero (M-003), Timestamp faltante (M-999) y Nulos.

Mi Primer Pipeline con Apache Beam

Análisis de Diseño de Datos - 5 pasos críticos

1. Mapeo de Atributos y Tipos de Datos (Schema Design)

Antes de programar, deben conocer su materia prima. El objetivo es definir el **contrato** de entrada para evitar errores de ejecución.

- **Análisis:** Identificar qué campos son obligatorios y cuáles pueden venir nulos (NULL). El CSV entrega todo como texto. ¿A qué debemos convertirlo?
- **Acción:** Definir el Casting.
 - **id_medidor:** String (Normalizar a minúsculas para uniformidad).
 - **consumo_kwh:** Float (Crítico para el filtro de > 10.0).
 - **voltaje:** Integer (Para futuras reglas de estabilidad).
 - **timestamp:** ISO-8601 String (Validar que no venga vacío).

Mi Primer Pipeline con Apache Beam

Análisis de Diseño de Datos - 5 pasos críticos

2. Definición del Grafo de Ejecución (DAG)

Beam no corre línea por línea como un script de Python normal; construye un Grafo Dirigido Acíclico.

- **Análisis:** ¿Cuántas veces leeremos el archivo?
- **Diseño:** Una sola lectura – > Una sola limpieza – > Triple Ramificación:
 - Rama A: Alertas de Sobrecarga.
 - Rama B: Detección de Anomalías (Consumo 0).
 - Rama C: Normalización para Almacenamiento Silver.
- **Objetivo:** Visualizar si el flujo es eficiente o si están repitiendo pasos innecesariamente.

Mi Primer Pipeline con Apache Beam

Análisis de Diseño de Datos - 5 pasos críticos

3. Estrategia de Manejo de Errores (Error Handling) (1/2)

- Decidir qué hacer cuando los datos no cumplen el estándar (Calidad de Datos).
- En procesamiento distribuido, una sola fila mal formateada puede 'romper' un worker si no se maneja bien.

Mi Primer Pipeline con Apache Beam

Análisis de Diseño de Datos - 5 pasos críticos

3. Estrategia de Manejo de Errores (Error Handling) (2/2)

- **Análisis:** ¿Qué pasa si consumo_kwh trae un 'NULL' o una letra?
- **Decisión Técnica:** Implementar un bloque try/except en el primer Map.
 - **Acción:** Los registros corruptos se descartan (retornando None) y se filtran inmediatamente para que no 'contaminen' las ramas de Alertas o Silver.

Mi Primer Pipeline con Apache Beam

Análisis de Diseño de Datos - 5 pasos críticos

4. El Contrato de Salida (Sinking Strategy) (1/2)

- Definir cómo y dónde se guardan los resultados según su uso posterior.
- No es solo guardar un archivo; es cómo se guarda para que sea útil después.
- **Sharding:** en sistemas distribuidos, el resultado no es un archivo único, sino varios pedazos (shards) escritos en paralelo. Deben decidir cuántos archivos de salida quieren generar según el volumen.

Mi Primer Pipeline con Apache Beam

Análisis de Diseño de Datos - 5 pasos críticos

4. El Contrato de Salida (Sinking Strategy) (2/2)

- **Análisis:** ¿Cómo deben ser los archivos de salida?
- **Especificación:**
 - **Alertas:** Archivo .txt plano (fácil de integrar con sistemas de SMS/Email).
 - **Anomalías:** Archivo .json (estructurado para el equipo de soporte técnico).
 - **Silver:** Archivo .csv o .parquet particionado (para el equipo de BI/Facturación).

Mi Primer Pipeline con Apache Beam

Análisis de Diseño de Datos - 5 pasos críticos

5. Aplicación del Modelo de las 4 Preguntas

Aterrizar la lógica de negocio al modelo de procesamiento.

- **What (¿Qué?):**

Mi Primer Pipeline con Apache Beam

Análisis de Diseño de Datos - 5 pasos críticos

5. Aplicación del Modelo de las 4 Preguntas

Aterrizar la lógica de negocio al modelo de procesamiento.

- **What (¿Qué?):** Filtrar excesos de carga, detectar ceros y normalizar IDs.

Mi Primer Pipeline con Apache Beam

Análisis de Diseño de Datos - 5 pasos críticos

5. Aplicación del Modelo de las 4 Preguntas

Aterrizar la lógica de negocio al modelo de procesamiento.

- **What (¿Qué?):** Filtrar excesos de carga, detectar ceros y normalizar IDs.
- **Where (¿Dónde?):**

Mi Primer Pipeline con Apache Beam

Análisis de Diseño de Datos - 5 pasos críticos

5. Aplicación del Modelo de las 4 Preguntas

Aterrizar la lógica de negocio al modelo de procesamiento.

- **What (¿Qué?):** Filtrar excesos de carga, detectar ceros y normalizar IDs.
- **Where (¿Dónde?):** En esta etapa es Batch (procesamos el archivo completo del día).

Mi Primer Pipeline con Apache Beam

Análisis de Diseño de Datos - 5 pasos críticos

5. Aplicación del Modelo de las 4 Preguntas

Aterrizar la lógica de negocio al modelo de procesamiento.

- **What (¿Qué?):** Filtrar excesos de carga, detectar ceros y normalizar IDs.
- **Where (¿Dónde?):** En esta etapa es Batch (procesamos el archivo completo del día).
- **When (¿Cuándo?):**

Mi Primer Pipeline con Apache Beam

Análisis de Diseño de Datos - 5 pasos críticos

5. Aplicación del Modelo de las 4 Preguntas

Aterrizar la lógica de negocio al modelo de procesamiento.

- **What (¿Qué?):** Filtrar excesos de carga, detectar ceros y normalizar IDs.
- **Where (¿Dónde?):** En esta etapa es Batch (procesamos el archivo completo del día).
- **When (¿Cuándo?):** El resultado se emite al finalizar el procesamiento del lote.

Mi Primer Pipeline con Apache Beam

Análisis de Diseño de Datos - 5 pasos críticos

5. Aplicación del Modelo de las 4 Preguntas

Aterrizar la lógica de negocio al modelo de procesamiento.

- **What (¿Qué?):** Filtrar excesos de carga, detectar ceros y normalizar IDs.
- **Where (¿Dónde?):** En esta etapa es Batch (procesamos el archivo completo del día).
- **When (¿Cuándo?):** El resultado se emite al finalizar el procesamiento del lote.
- **How (¿Cómo?):**

Mi Primer Pipeline con Apache Beam

Análisis de Diseño de Datos - 5 pasos críticos

5. Aplicación del Modelo de las 4 Preguntas

Aterrizar la lógica de negocio al modelo de procesamiento.

- **What (¿Qué?):** Filtrar excesos de carga, detectar ceros y normalizar IDs.
- **Where (¿Dónde?):** En esta etapa es Batch (procesamos el archivo completo del día).
- **When (¿Cuándo?):** El resultado se emite al finalizar el procesamiento del lote.
- **How (¿Cómo?):** Los datos se acumulan en el almacenamiento final reemplazando o añadiendo al histórico del día.

Mi Primer Pipeline con Apache Beam

Código

- https://github.com/ctorresf/developer-guides/blob/main/apache_beam/vscode-devcontainer-basic-beam-examples/src/simple_pipeline_read_from_csv.py

Más allá del Map

El límite de beam.Map y beam.Filter

- **Map:** Recibe 1 elemento, devuelve exactamente 1 elemento.
- **Filter:** Recibe 1 elemento, devuelve 1 o 0 (pasa o no pasa).
- **El Problema:** ¿Qué pasa si de una fila de log queremos generar 3 alertas distintas? ¿O si necesitamos lógica compleja con variables externas?

La Solución: ParDo (Parallel Do)

- Es el motor de procesamiento más versátil de Beam.
- Permite ejecutar lógica arbitraria en paralelo sobre cada elemento de una PCollection.

El Corazón de Beam - DoFn

Para usar un ParDo, necesitamos definir una clase DoFn (Dataflow Function).

Anatomía de un DoFn:

- **Clase:** Hereda de beam.DoFn.
- **Método process:** Es donde ocurre la magia.
- **Yield:** A diferencia de return, yield permite devolver 0, 1 o muchos elementos para un solo dato de entrada.

DoFn

Ejemplo

```
class ClasificadorEnergia (beam.DoFn):  
    def process(self , elemento):  
        if elemento['kwh'] > 10:  
            yield beam.pvalue.TaggedOutput(  
                                                        'alerta ', elemento)  
        if elemento['kwh'] == 0:  
            yield beam.pvalue.TaggedOutput(  
                                                        'revision ', elemento)  
        yield elemento # Flujo principal
```

Joins en Beam

¿Cómo unir dos PCollections?

- En un sistema distribuido, no podemos simplemente 'buscar' en otra tabla.
- Necesitamos que los datos que comparten la misma llave (Key) viajen a la misma máquina al mismo tiempo.
- El Operador: **CoGroupByKey**
 - Es el equivalente al JOIN.
 - Agrupa elementos de múltiples PCollections que comparten la misma llave.

Tipos de Joins y Side Inputs

Dependiendo del tamaño de los datos, un Ingeniero de Datos elige dos estrategias:

1. CoGroupByKey (Join Pesado)

- **Uso:** Cuando ambas colecciones son grandes (Big Data vs. Big Data).
- **Mecánica:** Beam realiza un "Shuffle" (mueve datos por la red) para agrupar todo lo que tenga el mismo ID.
- **Costo:** Alto uso de red y memoria.

Tipos de Joins y Side Inputs

Dependiendo del tamaño de los datos, un Ingeniero de Datos elige dos estrategias:

2. Side Inputs (Join Ligero)

- **Uso:** Cuando quieres unir un flujo gigante (millones de lecturas) con una tabla pequeña que cabe en memoria (ej. una lista de 500 sucursales).
- **Mecánica:** La tabla pequeña se envía como un "Diccionario.^a todos los workers. No hay Shuffle.
- **Costo:** Muy eficiente. Se usa dentro de un ParDo.

Tiempo en Streaming

Conceptos

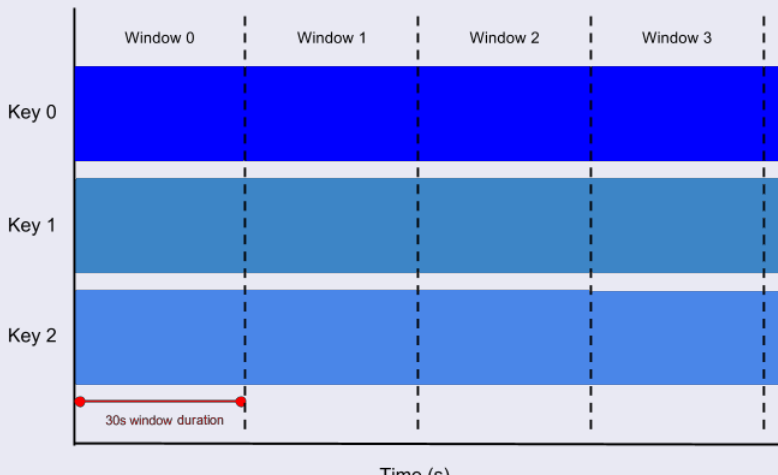
- **Event Time (Tiempo del Evento):** Cuándo ocurrió el hecho en el mundo real (ej. 18:00:00). Está pegado al dato.
- **Processing Time (Tiempo de Procesamiento):** Cuándo llega el dato al servidor de Google Cloud o AWS (ej. 18:00:05).
- **El desfase (Skew):** La diferencia entre ambos. Es causado por latencia de red, túneles (en camiones) o congestión de sensores.

Windowing - Cortando el Tiempo

Tipo de Ventana	Comportamiento	Caso de Uso
Fixed (Fijas)	Segmentos de tiempo estáticos (ej. cada 5 min).	Reportes de consumo por hora.
Sliding (Deslizantes)	Ventanas que se solapan (ej. últimos 5 min, cada 1 min).	Ritmo cardíaco promedio de los últimos 30s, actualizado cada 1s.
Session (Sesión)	Basadas en actividad/inactividad.	Análisis de uso de una App móvil.

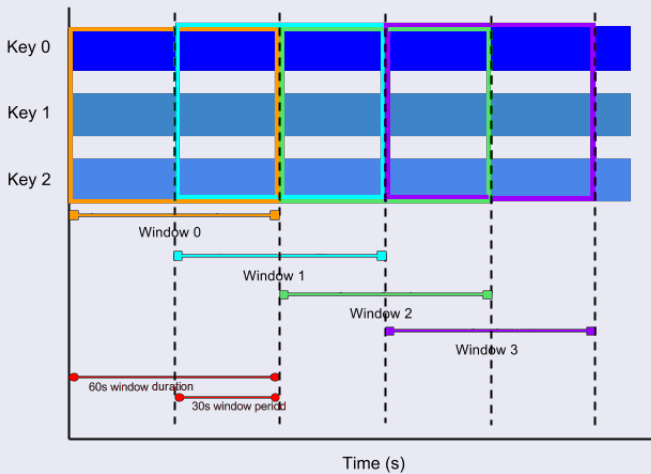
Windowing - Cortando el Tiempo

Fixed time windows



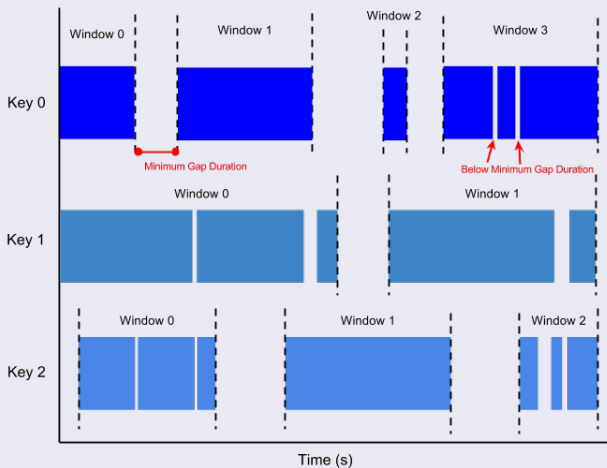
Windowing - Cortando el Tiempo

Sliding time windows



Windowing - Cortando el Tiempo

Session windows



Watermarks y Triggers

¿Cómo saber cuándo cerrar una ventana si los datos vienen atrasados?

- **Watermark:**

- Es la 'línea de marea'.
- Es la estimación del sistema sobre qué tan atrasados están los datos.
- Si el Watermark pasa las 10:00, la ventana de las 10:00 se considera 'lista'.

- **Triggers (Disparadores):** Dictan cuándo emitir el resultado.

- **Early Triggers:** Emitir promedios parciales antes de cerrar la ventana.
- **Late Triggers:** Emitir una corrección si un dato llegó después del Watermark.

Caso Práctico: Monitoreo de Carga en Tiempo Real

Escenario

- La red eléctrica de Santiago está sufriendo micro-cortes.
- Necesitamos detectar si el promedio de voltaje de un transformador cae por debajo de 200V en un **lapso de 1 minuto**, calculado **cada 30 segundos**, para enviar una cuadrilla técnica antes de que el sector se quede sin luz.

Caso Práctico: Monitoreo de Carga en Tiempo Real

Los 5 Pasos de Análisis (Diseño de Streaming)

1. Mapeo de Atributos y Timestamps (Event Time)

- En streaming, el atributo más importante es el Timestamp del sensor.
- **Payload:** { "id_trans": "T-100", "voltaje": 198, "ts": "2026-04-03T19:00:05Z" }.
- **Acción:** Debemos extraer el campo ts y convertirlo en un objeto Timestamp de Beam para que el motor sepa en qué 'momento del tiempo real' ocurrió la lectura, ignorando la hora del servidor.

Caso Práctico: Monitoreo de Carga en Tiempo Real

Los 5 Pasos de Análisis (Diseño de Streaming)

2. Definición del DAG de Ventanas

El flujo ya no es lineal, es un flujo continuo que se 'rebalsa' en ventanas.

- **Nodo 1:** Ingesta de Stream (Pub/Sub, Kafka, RabbitMQ, etc).
- **Nodo 2:** Asignación de Event Time.
- **Nodo 3:** Aplicación de Ventana Deslizante (Sliding Window).
- **Nodo 4:** Agregación (Promedio por Llave).
- **Nodo 5:** Disparador de Alerta (Filtro $< 200V$).

Caso Práctico: Monitoreo de Carga en Tiempo Real

Los 5 Pasos de Análisis (Diseño de Streaming)

3. Estrategia de Datos Tardíos (Watermark & Late Data)

¿Qué hacemos si un sensor tiene mala señal y envía su lectura de las 19:00 a las 19:05?

- **Análisis:** Definir un Allowed Lateness (Latencia Permitida).
- **Decisión:** Permitiremos datos con hasta 2 minutos de retraso. Si llegan después, se descartan para no alterar los promedios ya reportados a la central de energía.

Caso Práctico: Monitoreo de Carga en Tiempo Real

Los 5 Pasos de Análisis (Diseño de Streaming)

4. Contrato de Salida (Sink Strategy)

El resultado debe ser instantáneo.

- **Destino A (Alertas):** Escritura en un log de consola o base de datos de alta velocidad para notificaciones push.
- **Destino B (Dashboard):** Envío a una tabla de BigQuery para visualización en vivo del estado de la red.

Caso Práctico: Monitoreo de Carga en Tiempo Real

Los 5 Pasos de Análisis (Diseño de Streaming)

5. El Modelo de las 4 Preguntas (Streaming Edition)

- **What (¿Qué?):**

Caso Práctico: Monitoreo de Carga en Tiempo Real

Los 5 Pasos de Análisis (Diseño de Streaming)

5. El Modelo de las 4 Preguntas (Streaming Edition)

- **What (¿Qué?):** Promedio de voltaje por transformador.

Caso Práctico: Monitoreo de Carga en Tiempo Real

Los 5 Pasos de Análisis (Diseño de Streaming)

5. El Modelo de las 4 Preguntas (Streaming Edition)

- **What (¿Qué?):** Promedio de voltaje por transformador.
- **Where (¿Dónde?):**

Caso Práctico: Monitoreo de Carga en Tiempo Real

Los 5 Pasos de Análisis (Diseño de Streaming)

5. El Modelo de las 4 Preguntas (Streaming Edition)

- **What (¿Qué?):** Promedio de voltaje por transformador.
- **Where (¿Dónde?):** Ventana deslizante de 60 segundos, con deslizamiento cada 30 segundos.

Caso Práctico: Monitoreo de Carga en Tiempo Real

Los 5 Pasos de Análisis (Diseño de Streaming)

5. El Modelo de las 4 Preguntas (Streaming Edition)

- **What (¿Qué?):** Promedio de voltaje por transformador.
- **Where (¿Dónde?):** Ventana deslizante de 60 segundos, con deslizamiento cada 30 segundos.
- **When (¿Cuándo?):**

Caso Práctico: Monitoreo de Carga en Tiempo Real

Los 5 Pasos de Análisis (Diseño de Streaming)

5. El Modelo de las 4 Preguntas (Streaming Edition)

- **What (¿Qué?):** Promedio de voltaje por transformador.
- **Where (¿Dónde?):** Ventana deslizante de 60 segundos, con deslizamiento cada 30 segundos.
- **When (¿Cuándo?):** Al cierre del Watermark (cuando el sistema cree que ya no vienen más datos de ese minuto).

Caso Práctico: Monitoreo de Carga en Tiempo Real

Los 5 Pasos de Análisis (Diseño de Streaming)

5. El Modelo de las 4 Preguntas (Streaming Edition)

- **What (¿Qué?):** Promedio de voltaje por transformador.
- **Where (¿Dónde?):** Ventana deslizante de 60 segundos, con deslizamiento cada 30 segundos.
- **When (¿Cuándo?):** Al cierre del Watermark (cuando el sistema cree que ya no vienen más datos de ese minuto).
- **How (¿Cómo?):**

Caso Práctico: Monitoreo de Carga en Tiempo Real

Los 5 Pasos de Análisis (Diseño de Streaming)

5. El Modelo de las 4 Preguntas (Streaming Edition)

- **What (¿Qué?):** Promedio de voltaje por transformador.
- **Where (¿Dónde?):** Ventana deslizante de 60 segundos, con deslizamiento cada 30 segundos.
- **When (¿Cuándo?):** Al cierre del Watermark (cuando el sistema cree que ya no vienen más datos de ese minuto).
- **How (¿Cómo?):** Resultados acumulados por ventana.

Caso Práctico: Monitoreo de Carga en Tiempo Real

Código

- https://github.com/ctorresf/developer-guides/blob/main/apache_beam/vscode-devcontainer-basic-beam-examples/src/sliding_time_window_example.py

Clase 2: ETL con Python

Módulo 7: Data Pipelines y plataformas de visualización

Claudio Torres Fonseca

mail: `claudio.torresf@usm.cl`

Universidad de Santiago de Chile

Diplomado en Data Engineer
2026